

Sliding Window

A complete, practical study guide — from first principles to LC 239. Written for someone who has already mastered Two Pointers.

1. Sliding Window Concept
2. Fixed-Size Sliding Window
3. Variable-Size Sliding Window
4. Longest Valid Window (LC 3)
5. At Most K (LC 904)
6. Minimum Valid Window (LC 209)
7. Frequency Map Windows (LC 567, LC 438)
8. Character Replacement (LC 424)
9. Exactly K (LC 992)
10. Minimum Window Substring (LC 76)
11. Advanced: Sliding Window Maximum (LC 239)
12. Common Mistakes
13. Pattern Recognition Table
14. Practice Problems (15 problems)
15. Final Revision Templates

1. Sliding Window — Concept

What it is

Sliding Window is a technique for scanning a contiguous range (a "window") over an array or string using two indices, `left` and `right`, instead of recomputing a result from scratch for every possible subarray. The window grows by moving `right` forward and shrinks by moving `left` forward. At every point, the window `[left, right]` represents "the current candidate subarray/substring", and you maintain some running state (sum, count, frequency map) that updates in $O(1)$ as the window moves.

Why it works

The brute-force way to check all contiguous subarrays is $O(n^2)$ or $O(n^3)$: pick a start, pick an end, then recompute a sum/count over that range. Sliding Window exploits one fact: when you move from window `[l, r]` to `[l, r+1]` or `[l+1, r]`, the new state differs from the old state by only one element entering or leaving. So instead of recomputing over the whole range, you do an $O(1)$ update (add the new element, remove the old element). Total work becomes $O(n)$ because `left` and `right` each move forward at most n times across the whole algorithm — they never move backward.

Connection to Two Pointers

Sliding Window *is* a specialized form of Two Pointers. In Two Pointers, you typically had pointers moving toward each other (opposite ends) or at independent paces over a *sorted* structure, comparing individual elements — for example, checking pair sums or merging two sorted arrays. Sliding Window keeps both pointers moving in the **same direction** (left-to-right), and instead of comparing two elements, you maintain a **range** between them and track aggregate state (sum, count, distinct characters) over that whole range. Think of it as: Two Pointers reasons about "two positions", Sliding Window reasons about "everything between two positions".

When to recognize it

Reach for Sliding Window when a problem talks about a **contiguous** range and asks for one of these:

- "Longest / shortest subarray or substring that satisfies condition X"
- "Number of subarrays/substrings with exactly / at most K of something"
- "Maximum sum/average of a subarray of size K" (fixed size)
- "Contains all characters of..." / "is a permutation/anagram of..."
- The condition changes **monotonically** as the window grows — adding elements can only make the condition worse (or only better), never flip back and forth unpredictably. This monotonic property is what allows `left` to move forward and never backward.

If the problem needs a non-contiguous selection (e.g. subsequences, pick any elements), Sliding Window does not apply — that is usually DP or backtracking territory.

Subarray vs Substring

These are the same concept applied to different data types. A **subarray** is a contiguous slice of an array (numbers). A **substring** is a contiguous slice of a string (characters). Every technique in this guide applies identically to both — the window logic doesn't care whether the elements are integers or characters. LC 209 (subarray, numbers) and LC 3 (substring, characters) use the exact same skeleton.

Fixed vs Variable Window

Fixed-size window	Variable-size window
Window length is given (e.g. "subarray of size k")	Window length is not given — it grows and shrinks based on a condition
<code>right</code> and <code>left</code> both move every step, keeping <code>right - left + 1 == k</code> always	<code>right</code> always moves forward; <code>left</code> moves only when the window becomes invalid (or to minimize it)
Example: LC 643 (max average subarray of size k)	Example: LC 3, LC 209, LC 76

2. Fixed-Size Sliding Window

General template

```
def fixed_window(nums, k):
    window_val = 0
    # build the first window
    for i in range(k):
        window_val += nums[i]

    best = window_val
    for right in range(k, len(nums)):
        window_val += nums[right]      # add new element entering window
        window_val -= nums[right - k]  # remove element leaving window
        best = max(best, window_val)

    return best
```

Detailed explanation

A fixed-size window keeps its length constant at `k` for the entire scan. There are two phases:

- 1. **Build phase** — compute the state (sum, count, etc.) for the very first window, indices `0` to `k-1`, using a simple loop. This is the only place you "pay" for `k` elements.
- 2. **Slide phase** — for every position after that, the window shifts one step to the right. Exactly one element leaves (the one at index `right - k`, i.e. the leftmost element of the previous window) and exactly one element enters (`nums[right]`). You update the running state in $O(1)$: add the new, subtract the old. You never re-sum the whole window.

Because `left` is implicit here (it's always `right - k + 1`), you don't even need a separate `left` variable — you can derive it from `right`. That's what makes fixed-size windows the simplest case.

Dry run

nums = [1, 3, 2, 6, -1, 4, 1, 8, 2], k = 5 — find the max sum of any subarray of size 5.

Step	Window (indices)	Action	window_val	best
build	[0..4] = 1,3,2,6,-1	sum first 5	11	11
right=5	[1..5] = 3,2,6,-1,4	+4 (enter), -1 (leave idx0)	14	14
right=6	[2..6] = 2,6,-1,4,1	+1 (enter), -3 (leave idx1)	12	14
right=7	[3..7] = 6,-1,4,1,8	+8 (enter), -2 (leave idx2)	18	18
right=8	[4..8] = -1,4,1,8,2	+2 (enter), -6 (leave idx3)	13	18

Answer: 18.

Example: LC 643 — Maximum Average Subarray I

Given an array and integer `k`, find the maximum average value of any contiguous subarray of length `k`.

Brute force

```
def findMaxAverage_brute(nums, k):
    n = len(nums)
    best = float('-inf')
    for i in range(n - k + 1):
        s = sum(nums[i:i+k])      # O(k) recompute every time
        best = max(best, s)
    return best / k
```

For every starting index `i`, this re-sums `k` elements from scratch. That's $O(n)$ windows $\times$ $O(k)$ work each = **$O(n \cdot k)$** .

Optimized (sliding window)

```
def findMaxAverage(nums, k):
    window_sum = sum(nums[:k])
    best = window_sum

    for right in range(k, len(nums)):
        window_sum += nums[right] - nums[right - k]
        best = max(best, window_sum)

    return best / k
```

Time and space complexity

Brute force: Time $O(n \cdot k)$, Space $O(1)$

Optimized: Time $O(n)$ — each index visited once during build + once during slide, Space $O(1)$ — only a running sum is kept

3. Variable-Size Sliding Window

General template

```
def variable_window(nums):
    left = 0
    state = 0          # running sum / count / freq map
    best = 0           # or float('inf') for "minimum" problems

    for right in range(len(nums)):
        state += nums[right]          # 1. expand: include nums[right]

        while window_is_invalid(state): # 2. shrink while broken
            state -= nums[left]
            left += 1

        best = update(best, right, left) # 3. record answer (only on valid window)

    return best
```

Expand and shrink

Variable windows work in two alternating motions:

- **Expand:** `right` moves forward exactly once per outer loop iteration, unconditionally. This is how you "try" including one more element.
- **Shrink:** `left` moves forward zero or more times, but only in response to the window becoming invalid (too big a sum, too many distinct characters, a duplicate, etc). Shrinking restores the invariant before you record an answer.

Across the entire run, `right` makes exactly `n` moves and `left` makes at most `n` moves (it never exceeds `right` and never moves backward), so total pointer movement is $O(n)$, not $O(n^2)$.

Why we use `while` instead of `if`

A single bad element can require shrinking by more than one position to fix. If you used `if`, you'd only remove one element from the left and then move on — even if the window is still invalid. Consider "longest substring with at most 2 distinct characters": if the window currently has 3 distinct characters and you remove one element from the left, it's entirely possible the removed element wasn't the "extra" distinct character (there could be duplicates of it still remaining ahead), so the window could still have 3 distinct characters after one shrink. `while` guarantees you keep shrinking **until the invariant is restored**, no matter how many steps that takes. `if` only checks the condition once and silently leaves you with an invalid window, producing wrong answers.

Window invariant

The "invariant" is the condition you promise is true about the window every time you're about to record an answer — e.g. "sum $\leq$ target", "at most `k` distinct characters", "no duplicate characters". The `while` loop's entire job is to restore this invariant after each expansion. You should be able to state your invariant in one sentence before writing any code; if you can't, you haven't understood the problem's shrink condition yet.

Detailed dry run

nums = [2, 1, 5, 2, 3, 2], target = 7 — find length of smallest subarray with sum $\geq$ target (invariant to restore: sum < target after each shrink is not the goal — here we shrink *while valid* to minimize; see Section 6 for the full walkthrough). We show the expand/shrink mechanics only:

right	add nums[right]	window sum	shrink?	left after
0	+2	2	no (2<7)	0
1	+1	3	no	0
2	+5	8	yes: 8 $\geq$ 7 $\rightarrow$ shrink: -2 $\rightarrow$ sum=6, left=1	1
3	+2	8	yes: shrink -1 $\rightarrow$ sum=7, left=2; still $\geq$ 7 $\rightarrow$ shrink -5 $\rightarrow$ sum=2, left=3	3
4	+3	5	no	3
5	+2	7	yes: shrink -2 $\rightarrow$ sum=5, left=4	4

Notice at `right=3`, one expansion required **two** shrink steps — exactly why `while` is required instead of `if`.

4. Longest Valid Window — LC 3

Longest Substring Without Repeating Characters

Given a string `s`, find the length of the longest substring without repeating characters.

Explain set usage

We need to know, at any moment, whether the character we're about to add already exists inside the current window. A **set** gives $O(1)$ membership checks and $O(1)$ insert/remove, which is exactly what we need to test "is this character already in my window?" and to keep the window's contents in sync as it slides. The invariant is: *the set never contains a duplicate*. Whenever adding `s[right]` would create a duplicate, we shrink from the left — removing characters from both the set and the window — until the duplicate is gone.

Full Python code

```
def lengthOfLongestSubstring(s: str) -> int:
    char_set = set()
    left = 0
    best = 0

    for right in range(len(s)):
        while s[right] in char_set:
            char_set.remove(s[left])
            left += 1
        char_set.add(s[right])
        best = max(best, right - left + 1)

    return best
```

Line-by-line explanation

- `char_set = set()` — holds exactly the characters currently inside the window `[left, right]`.
- `left = 0` — left boundary of the window, starts unmoved.
- `best = 0` — tracks the longest valid window length seen so far.
- `for right in range(len(s))` — try extending the window by one character at a time; this is the "expand" step, it always runs.
- `while s[right] in char_set` — the character we're about to add is already present, meaning adding it would create a duplicate. We must shrink until it's safe to add.
- `char_set.remove(s[left]); left += 1` — remove the leftmost character from both the logical window and the set, then advance `left`. This repeats until `s[right]` is no longer in the set.
- `char_set.add(s[right])` — now it's safe; officially include `s[right]` in the window.
- `best = max(best, right - left + 1)` — the window is guaranteed valid here (no duplicates), so record its length. `right - left + 1` is the count of elements between the two indices inclusive.

Complete dry run

s = "abcabcbb"						
right	s[right]	in set?	shrink	set after	left	best
0	a	no	—	{a}	0	1
1	b	no	—	{a,b}	0	2
2	c	no	—	{a,b,c}	0	3
3	a	yes	remove s[0]='a', left→1	{b,c,a}	1	3
4	b	yes	remove s[1]='b', left→2	{c,a,b}	2	3
5	c	yes	remove s[2]='c', left→3	{a,b,c}	3	3
6	b	yes	remove s[3]='a'→still has b; remove s[4]='b', left→5	{c,b}	5	3
7	b	yes	remove s[5]='c'→still has b; remove s[6]='b', left→7	{b}	7	3

Answer: **3** (from "abc").

Time: $O(n)$ — each character added and removed from the set at most once. **Space:** $O(\min(n, \text{alphabet size}))$ for the set.

5. At Most K

Frequency map

"At most K" problems (e.g. at most K distinct characters, at most K odd numbers) need to know *how many kinds* of something are in the window, or how many times each value occurs. A **frequency map** (dict / `Counter`) maps each value to its count in the current window. The number of distinct keys with count > 0 tells you "how many kinds" are present. When a count drops to 0 on removal, delete the key — this keeps `len(freq_map)` an accurate distinct-count.

Example: LC 904 — Fruit Into Baskets

You have two baskets, each can hold only one type of fruit, but can hold any amount. Find the length of the longest subarray containing at most 2 distinct fruit types. This is literally "longest substring with at most K distinct" with K = 2.

Detailed explanation

Expand the window by adding `fruits[right]` to the frequency map. If the map now has more than 2 distinct keys, the window is invalid — shrink from the left, decrementing counts and deleting zero-count keys, until only 2 distinct types remain. Because this is a "longest" problem, we record `best` **after** the shrink loop, i.e. only when the window is guaranteed valid.

Code

```
def totalFruit(fruits: list[int]) -> int:
    count = {}
    left = 0
    best = 0

    for right in range(len(fruits)):
        count[fruits[right]] = count.get(fruits[right], 0) + 1

        while len(count) > 2:
            count[fruits[left]] -= 1
            if count[fruits[left]] == 0:
                del count[fruits[left]]
            left += 1

        best = max(best, right - left + 1)

    return best
```

Dry run

fruits = [1, 2, 1, 2, 3, 3, 1]					
right	fruit	count map	distinct>2?	left	best
0	1	{1:1}	no	0	1
1	2	{1:1,2:1}	no	0	2
2	1	{1:2,2:1}	no	0	3
3	2	{1:2,2:2}	no	0	4
4	3	{1:2,2:2,3:1}	yes → shrink left=0 (fruit 1): {1:1,2:2,3:1}, left→1, still 3 distinct → shrink left=1 (fruit 2): {1:1,2:1,3:1}, left→2, still 3 → shrink left=2 (fruit1): {2:1,3:1}, left→3	3	4
5	3	{2:1,3:2}	no	3	4
6	1	{2:1,3:2,1:1}	yes → shrink left=3 (fruit2): {3:2,1:1}, left→4	4	max(4, 6-4+1=3)=4
Answer: 4.					

Time: O(n). **Space:** O(k) for the frequency map (at most 3 keys ever exist before a shrink resolves it).

6. Minimum Valid Window — LC 209

Minimum Size Subarray Sum

Given a positive-integer array `nums` and a target, find the length of the shortest contiguous subarray whose sum is $\geq$ target. Return 0 if none exists.

Why positive numbers matter

Sliding Window's correctness for "shrink while valid" depends on the window sum being **monotonic**: adding an element must only increase the sum, and removing one must only decrease it. That's guaranteed only when every number is positive. If negative numbers were allowed, shrinking the window (removing the leftmost element) might *decrease* the sum in a way that makes an already-invalid window valid again unpredictably, or growing the window might not actually make progress toward the target — the whole "left never needs to move backward" guarantee breaks. That's why LC 209 explicitly uses positive integers; with negatives, you need a different approach (e.g. prefix sums + monotonic deque, or binary search doesn't even work reliably).

Code

```
def minSubArrayLen(target: int, nums: list[int]) -> int:
    left = 0
    window_sum = 0
    best = float('inf')

    for right in range(len(nums)):
        window_sum += nums[right]

        while window_sum >= target:
            best = min(best, right - left + 1)
            window_sum -= nums[left]
            left += 1

    return best if best != float('inf') else 0
```

Note the shape difference from "longest" problems: here we record `best` **inside** the `while` loop, before shrinking — because we want the shortest window *while it's still valid*, and shrinking one more step might break validity. In "longest" problems we record *after* the while loop because we want the window at its largest valid size.

Detailed dry run

target = 7, nums = [2, 3, 1, 2, 4, 3]						
right	add	sum	sum≥7?	action	left	best
0	2	2	no	—	0	inf
1	3	5	no	—	0	inf
2	1	6	no	—	0	inf
3	2	8	yes	best=min(inf,4)=4; sum-=2(idx0)→6, left→1	1	4
4	4	10	yes	best=min(4,4-1+1=4)=4; sum-=3(idx1)→7, left→2; still≥7 → best=min(4,4-2+1=3)=3; sum-=1(idx2)→6, left→3	3	3
5	3	9	yes	best=min(3,5-3+1=3)=3; sum-=2(idx3)→7, left→4; still≥7 → best=min(3,5-4+1=2)=2; sum-=4(idx4)→3, left→5	5	2
Answer: 2 (subarray [4,3]).						

Time: O(n). **Space:** O(1).

7. Sliding Window + Frequency Map

The core idea

Many problems ask: "does the window's character composition exactly match some target composition?" (a permutation, an anagram). The technique: build a frequency map of the target string once, then maintain a frequency map of the current fixed-size window, and compare them. Since target length is fixed, this is a **fixed-size window** problem wearing a frequency-map disguise.

Example: LC 567 — Permutation in String

Given `s1` and `s2`, return True if `s2` contains a permutation of `s1` (i.e. some substring of `s2` is an anagram of `s1`).

```
from collections import Counter

def checkInclusion(s1: str, s2: str) -> bool:
    n1, n2 = len(s1), len(s2)
    if n1 > n2:
        return False

    need = Counter(s1)
    window = Counter(s2[:n1])

    if window == need:
        return True

    for right in range(n1, n2):
        window[s2[right]] += 1
        left_char = s2[right - n1]
        window[left_char] -= 1
        if window[left_char] == 0:
            del window[left_char]
        if window == need:
            return True

    return False
```

This is exactly the fixed-window template from Section 2 — build the first window's Counter, then slide: add the new char, remove the char leaving. Comparing two `Counter` objects for equality is O(alphabet size), not O(n), so the loop stays efficient.

Example: LC 438 — Find All Anagrams in a String

Same idea as LC 567, but instead of returning True/False on first match, collect every starting index where the window's Counter equals `need`.

```
from collections import Counter

def findAnagrams(s: str, p: str) -> list[int]:
    n, m = len(s), len(p)
    result = []
    if m > n:
        return result

    need = Counter(p)
    window = Counter(s[:m])

    if window == need:
        result.append(0)

    for right in range(m, n):
        window[s[right]] += 1
        left_char = s[right - m]
        window[left_char] -= 1
        if window[left_char] == 0:
            del window[left_char]
        if window == need:
            result.append(right - m + 1)

    return result
```

Explain frequency counting clearly

The two maps answer two different questions: `need` answers "what composition am I looking for?" — it never changes. `window` answers "what composition do I currently have?" — it changes every slide. Equality of the two maps means the window is an exact multiset match: same characters, same counts, order irrelevant. Deleting zero-count keys from `window` is essential — otherwise `window == need` would fail even on a true match, because `Counter({'a': 0})` is not equal to a Counter missing the key `'a'` unless zero-entries are cleaned up (in Python's `Counter`, they actually do compare equal since missing keys default to 0 — but cleaning up keeps the map small and is good

practice, and is required if you're comparing dict sizes or distinct-key counts instead of full equality).

8. Character Replacement — LC 424

Longest Repeating Character Replacement

Given a string `s` and integer `k`, you may replace up to `k` characters in the string with any other character. Find the length of the longest substring you can make consist of a single repeated character.

The key inequality: `window_length - max_frequency <= k`

Inside any window, if the most frequent character appears `max_frequency` times, then every *other* character in the window needs to be replaced to make the whole window a single repeated character. The number of "other" characters is exactly `window_length - max_frequency`. The window is achievable with at most `k` replacements exactly when:

```
window_length - max_frequency <= k
```

Explain the reasoning carefully

Why track `max_frequency` and not recompute it exactly on every shrink? Because `max_frequency` only needs to be a valid **upper bound** for correctness of the final answer, not an exact live value. Here's why that's safe: the answer we want is the *longest* valid window ever seen. If `max_frequency` is stale (too high from an earlier, larger window), the inequality check becomes falsely lenient for a moment — but that only prevents us from shrinking the window, which means the window's length simply doesn't grow past what it already achieved at a larger true `max_frequency`. In other words, once `max_frequency` reaches some value `M`, the algorithm never reports a length greater than `M + k`, and it never needs to be re-lowered because we're only interested in the highest length ever achieved, not in producing a valid window at every single step. This lets us skip recomputing the max over the whole frequency map on every shrink, which would cost $O(26)$ each time (still technically $O(n)$ overall, but tracking the running max is simpler and just as correct).

Full code

```
def characterReplacement(s: str, k: int) -> int:
    count = {}
    left = 0
    max_freq = 0
    best = 0

    for right in range(len(s)):
        count[s[right]] = count.get(s[right], 0) + 1
        max_freq = max(max_freq, count[s[right]])

        while (right - left + 1) - max_freq > k:
            count[s[left]] -= 1
            left += 1

        best = max(best, right - left + 1)

    return best
```

Dry run

s = "AABABBA", k = 1							
right	char	count	max_freq	len-max_freq	>k?	left	best
0	A	{A:1}	1	1-1=0	no	0	1
1	A	{A:2}	2	2-2=0	no	0	2
2	B	{A:2,B:1}	2	3-2=1	no(1≤1)	0	3
3	A	{A:3,B:1}	3	4-3=1	no	0	4
4	B	{A:3,B:2}	3	5-3=2	yes → shrink left=0 'A': {A:2,B:2}, left→1	1	4
5	B	{A:2,B:3}	3(stale, true max now 3 for B — still 3)	5-3=2	yes → shrink left=1 'A': {A:1,B:3}, left→2	2	4
6	A	{A:2,B:3}	3	5-3=2	yes → shrink left=2 'B': {A:2,B:2}, left→3	3	max(4,6-3+1=4)=4
Answer: 4 (e.g. "AABA" → replace the B with A, or "ABBA" → replace one A).							

Time: $O(n)$. **Space:** $O(26) = O(1)$ for the count map.

9. Exactly K — LC 992

Subarrays with K Different Integers

Given an array of positive integers, count the number of contiguous subarrays that contain exactly `k` different integers.

The formula: `exactly(K) = atMost(K) - atMost(K-1)`

Sliding Window naturally answers "at most K" cleanly (Section 5's shrink-while-too-many pattern), because "at most" is monotonic: once a window has $\leq K$ distinct values, every shrunk sub-window of it also has $\leq K$ distinct values. "Exactly K" is **not** directly monotonic in the same way — a window with exactly K distinct values, when shrunk, might become K-1, K-2, etc., unpredictably; you can't cleanly shrink "while exactly K" because a valid window is bordered by both "too many" and "too few" — there is no single-direction invariant.

The fix: count subarrays with **at most** K distinct, and subtract the count of subarrays with **at most** K-1 distinct. What's left is exactly the subarrays with **exactly** K distinct, because:

{subarrays with $\leq K$ distinct} = {subarrays with exactly K distinct} $\cup$ {subarrays with $\leq K-1$ distinct}

These two sets on the right are disjoint, so subtracting isolates exactly the set we want.

Why we use `right - left + 1`

When counting subarrays ending at `right` that fit inside a valid (at-most-K) window starting anywhere from `left` to `right`, every subarray `[i, right]` for `i` in `[left, right]` is valid (fewer distinct values only, since shrinking left never adds distinct values). There are exactly `right - left + 1` such starting positions `i`. So instead of counting one subarray at a time, adding `right - left + 1` to the running total counts **all** valid subarrays ending at this `right` in one O(1) step — this is the standard "count subarrays" trick paired with at-most-K sliding window.

Full code

```
def subarraysWithKDistinct(nums: List[Int], k: Int) -> Int:
  def at_most_k(k: Int): Int =
    if k < 0:
      return 0
    count = Map.empty[Int, Int]
    left = 0
    total = 0
    for right in range(len(nums)):
      count[nums[right]] = count.get(nums[right], 0) + 1
      while len(count) > k:
        count[nums[left]] -= 1
        if count[nums[left]] == 0:
          del count[nums[left]]
        left += 1
      total += right - left + 1
    return total
  return at_most_k(k) - at_most_k(k - 1)
```

Dry run

nums = [1, 2, 1, 2, 3], k = 2
at_most_k(2):

right	val	map	left	right-left+1	total
0	1	{1:1}	0	1	1
1	2	{1:1,2:1}	0	2	3
2	1	{1:2,2:1}	0	3	6
3	2	{1:2,2:2}	0	4	10
4	3	{1:2,2:2,3:1} → shrink: remove idx0 '1'→{1:1,2:2,3:1}, left→1, still 3 distinct → remove idx1 '2'→{1:1,2:1,3:1}, left→2, still 3 → remove idx2 '1'→{2:1,3:1}, left→3	3	2	12

at_most_k(2) = 12.
at_most_k(1): (only single-value windows survive; work omitted for brevity) gives windows: [1],[2],[1],[2],[3] each contributing at their own right, plus small runs — result = 5.
Answer: `exactly(2) = 12 - 5 = 7`.

Time: O(n) — two O(n) passes. **Space:** O(k) for the frequency map.

10. Minimum Window Substring — LC 76

The problem

Given strings `s` and `t`, find the smallest substring of `s` that contains all characters of `t` (including duplicates — if `t` has two 'a's, the window needs at least two 'a's too). Return "" if no such window exists.

The frequency-map technique

Build a frequency map `need` from `t`. As you expand the window in `s`, maintain a frequency map `window` of characters currently included. Track a single counter `have` = "how many distinct characters currently meet or exceed their required count in `need`", compared against `required` = total distinct characters in `need`. The window is valid exactly when `have == required`. This avoids comparing entire maps on every step (which Section 7's problems could afford since window size was fixed and small; here the window size varies, so we want an O(1) validity check, not an O(alphabet) one).

Full Python solution

```
from collections import Counter

def minWindow(s: str, t: str) -> str:
    if not s or not t:
        return ""

    need = Counter(t)
    required = len(need)          # distinct chars we must satisfy

    window = {}
    have = 0                     # distinct chars currently satisfied
    left = 0
    best_len = float('inf')
    best_left = 0

    for right in range(len(s)):
        c = s[right]
        window[c] = window.get(c, 0) + 1

        if c in need and window[c] == need[c]:
            have += 1

        while have == required:
            if (right - left + 1) < best_len:
                best_len = right - left + 1
                best_left = left

            left_char = s[left]
            window[left_char] -= 1
            if left_char in need and window[left_char] < need[left_char]:
                have -= 1
            left += 1

    return "" if best_len == float('inf') else s[best_left:best_left + best_len]
```

Detailed dry run

```
s = "ADOBECODEBANC", t = "ABC"
need = {A:1, B:1, C:1}, required = 3.
```

right	char	have	have==required?	shrink actions	best window
0	A	1	no	—	—
1	D	1	no	—	—
2	O	1	no	—	—
3	B	2	no	—	—
4	E	2	no	—	—
5	C	3	yes	window="ADOBEC" (len 6) → record best_len=6,"ADOBEC"; shrink left=0 'A': window count A=0<1 → have→2, left→1	"ADOBEC"
6	O	2	no	—	"ADOBEC"
7	D	2	no	—	"ADOBEC"
8	E	2	no	—	"ADOBEC"
9	B	2 (B already had count 1, now 2, still ≥ need so have stays)	no	—	"ADOBEC"
10	A	3	yes	window=s[1..10]="DOBECODEBA" (len 10) — not better than 6; shrink: left=1 'D' (not in need) → left→2; still have==3 → left=2 'O' (not in need) → left→3; still have==3 → left=3 'B': count B 2→1, still ≥1 so have stays 3 → left→4; still have==3 → left=4 'E' not in need → left→5; still have==3 → left=5 'C': count C 1→0<1 → have→2, left→6	"ADOBEC" (unchanged, none beat len 6 yet)
11	N	2	no	—	"ADOBEC"
12	C	3	yes	window=s[6..12]="ODEBANC" (len 7) — not better; shrink: left=6 'O' not in need → left→7; have==3 → left=7 'D' not in need → left→8; have==3 → left=8 'E' not in need → left→9; have==3 → left=9 'B': count 1→0<1 → have→2, left→10; loop ends	still "ADOBEC" (len 6) — but actual optimal is "BANC" len 4, found next

Continuing carefully: at right=12 before have drops, the window at the moment `have==3` first holds was `s[6..12]`, and shrinking step-by-step also re-checks length at *each* valid state inside the while loop. Re-tracing precisely: after left advances to 9 the window is `s[9..12]`="BANC", length 4, still valid (`have==3`) — this length 4 is recorded as the new best **before** removing 'B' drops have to 2. So `best_len` updates to 4, best string "BANC", then removing `s[9]='B'` finally breaks validity (`have→2`) and the while loop exits.

Answer: **"BANC"**.

This dry run shows the subtlety of LC 76: the `while have == required` loop records the best answer **on every valid state** it passes through while shrinking, not just once — because shrinking one step at a time can pass through several still-valid, progressively shorter windows before finally breaking.

Time: $O(|s| + |t|)$. **Space:** $O(|s| + |t|)$ for the frequency maps.

11. Advanced Sliding Window — LC 239

Sliding Window Maximum

Given an array `nums` and window size `k`, return an array of the maximum value in each window of size `k` as it slides from left to right.

Why a monotonic deque is needed

A naive fixed window (Section 2's template) works great for sums, because sum has an inverse operation — you can subtract the leaving element. **Max has no inverse.** If the maximum leaves the window, you cannot "undo" your way to the new maximum in $O(1)$ — you'd have to rescan the remaining window, which is $O(k)$ per step and $O(n \cdot k)$ overall.

The fix is a **monotonic deque** that stores *indices*, kept in decreasing order of their values, front to back. The front of the deque is always the index of the current window's maximum. Two rules keep it correct:

- **Before pushing a new index**, pop from the back every index whose value is $\leq$ the new value. Those popped elements can never be the maximum again — the new element is both later (will stay in the window longer) and at least as large, so they're strictly dominated and useless to keep.
- **Before reading the max**, pop from the front any index that has fallen outside the window (`index <= right - k`). Those are stale — the window no longer includes them.

Because each index is pushed once and popped at most once, total deque operations are $O(n)$, giving $O(1)$ amortized max-lookup per window.

Full code

```
from collections import deque

def maxSlidingWindow(nums: list[int], k: int) -> list[int]:
    dq = deque() # stores indices, values decreasing front-back
    result = []

    for right in range(len(nums)):
        # remove indices whose values are dominated by nums[right]
        while dq and nums[dq[-1]] <= nums[right]:
            dq.pop()
        dq.append(right)

        # remove front index if it's outside the window
        if dq[0] <= right - k:
            dq.popleft()

        # window fully formed once right >= k - 1
        if right >= k - 1:
            result.append(nums[dq[0]])

    return result
```

Detailed explanation

Trace the invariant carefully: at any point, `dq` holds candidate indices for "future window maximums", strictly decreasing in value from front to back, and strictly increasing in index (oldest at front). When you add `nums[right]`, anything smaller sitting at the back of the deque is now useless — it will never be the max while `nums[right]` is still in range, since `nums[right]` is both newer (stays longer) and bigger. That's why they get popped before insertion. Reading the front after evicting stale (out-of-window) indices always gives the true max in $O(1)$, because the deque is sorted by value and the front is both in-range and the largest.

12. Common Mistakes

Wrong window length

Using `right - left` instead of `right - left + 1`, or vice versa. The window `[left, right]` is inclusive on both ends, so it contains `right - left + 1` elements. Forgetting the `+1` silently undercounts every window by one — a common off-by-one that produces answers exactly 1 too small.

Moving left incorrectly

Advancing `left` before using its old value to update the state (e.g. incrementing `left` before subtracting `nums[left]` from the sum). Always update the state using the *current* `left` first, then increment. Also common: moving `left` more than one step per shrink iteration by accident, which skips checking the invariant on intermediate window sizes.

Forgetting to remove elements

When shrinking, some beginners advance `left` without updating the frequency map / sum / set at all — treating pointer movement as if it automatically forgets the element. It doesn't; you must explicitly decrement the count (or remove the set entry, or subtract from the sum) for `nums[left]` before moving on, or your state silently becomes stale and wrong.

Using `if` instead of `while`

As explained in Section 3, a single shrink step might not be enough to restore the invariant. Using `if` shrinks at most once per expansion, which is wrong whenever more than one element needs removing (e.g. Section 5's LC 904 dry run, `right=4`, needed two shrinks).

Updating the answer at the wrong time

For "longest" problems, record the answer *after* the shrink `while` loop (window is guaranteed valid then). For "shortest" problems, record the answer *inside* the shrink `while` loop, before each removal, because you want the smallest valid window, and once you shrink past validity you've lost the chance to measure that particular size. Swapping these two conventions produces subtly wrong answers that only show up on certain inputs.

Confusing "at most K" and "exactly K"

"At most K" is directly solvable with the standard shrink-while-too-many template because it's monotonic. "Exactly K" is not directly monotonic and needs the `atMost(K) - atMost(K-1)` trick (Section 9). Trying to write a direct "shrink while count != k" loop for exactly-K problems usually breaks, because a valid window can be both approached from above (too many) and below (too few), with no single shrink direction that always makes progress.

Using sliding window when negative numbers break the logic

Sum-based sliding window (like LC 209) relies on the sum increasing monotonically as the window grows and decreasing monotonically as it shrinks. Negative numbers break this: growing the window could decrease the sum, and shrinking could increase it, so "shrink while `sum ≥ target`" no longer guarantees you've found the minimal valid window, and might even cause you to shrink past a valid window incorrectly. If a problem allows negative numbers, look for prefix sums with a different structure (e.g. prefix sum + hash map, or prefix sum + monotonic deque) instead of a plain sliding window.

13. Pattern Recognition Table

Problem wording	Pattern	Data structure	When to shrink
"subarray of size k" / "average of every k elements"	Fixed size	running sum / running count	Every step (window length fixed) — add new, remove element leaving at <code>right - k</code>
"longest substring/subarray such that..."	Longest	set / frequency map / running sum	Only when window becomes invalid; record answer after shrink loop
"shortest/minimum substring/subarray such that..."	Shortest	running sum / frequency map with have/required counters	While window is still valid; record answer inside shrink loop, before each removal
"at most K distinct / at most K occurrences"	At most K	frequency map (dict of counts)	While number of distinct keys (or violating count) exceeds K
"exactly K distinct"	Exactly K	two at-most-K passes, frequency map in each	Compute <code>atMost(K) - atMost(K-1)</code> ; shrink rule is the "at most" rule inside each helper
"is a permutation/anagram of" / "contains all characters of"	Frequency matching	Counter / frequency map compared for equality, or have/required counters	Fixed window: always slide by 1. Variable window (LC 76 style): shrink while fully satisfied
"number of subarrays that satisfy..."	Counting subarrays	frequency map + running total using <code>right-left+1</code>	Shrink while invalid (at-most style), then add <code>right-left+1</code> to total each step

14. Practice Problems

15 problems, ordered by increasing difficulty, covering every pattern in this guide.

#	LC	Name	Difficulty	Pattern it teaches
1	643	Maximum Average Subarray I	Easy	Fixed-size window basics
2	1004	Max Consecutive Ones III	Medium	Longest window, at-most-K flips
3	3	Longest Substring Without Repeating Characters	Medium	Longest window with a set
4	209	Minimum Size Subarray Sum	Medium	Shortest window, positive-sum monotonicity
5	904	Fruit Into Baskets	Medium	At-most-K distinct with frequency map
6	1493	Longest Subarray of 1's After Deleting One Element	Medium	Longest window, at-most-1 zero
7	567	Permutation in String	Medium	Fixed window + frequency map equality
8	438	Find All Anagrams in a String	Medium	Fixed window + frequency map, collecting all matches
9	424	Longest Repeating Character Replacement	Medium	Longest window using the max-frequency inequality
10	713	Subarray Product Less Than K	Medium	Counting subarrays with <code>right-left+1</code>
11	992	Subarrays with K Different Integers	Hard	Exactly-K via <code>atMost(K)-atMost(K-1)</code>
12	76	Minimum Window Substring	Hard	Shortest window with have/required frequency matching
13	239	Sliding Window Maximum	Hard	Monotonic deque for $O(1)$ window max
14	1438	Longest Continuous Subarray With Absolute Diff $\leq$ Limit	Hard	Longest window with two monotonic deques (min & max)
15	632	Smallest Range Covering Elements from K Lists	Hard	Window over merged/sorted structure — sliding window generalized beyond a single array

15. Final Revision Page

Concise templates to memorize. Read these the night before an interview.

Fixed Window

```
window_val = sum(nums[:k])
best = window_val
for right in range(k, len(nums)):
    window_val += nums[right] - nums[right - k]
    best = max(best, window_val)
```

Longest Valid Window

```
left = 0
state = {}
best = 0
for right in range(len(nums)):
    add(state, nums[right])
    while invalid(state):
        remove(state, nums[left])
        left += 1
    best = max(best, right - left + 1)
```

Shortest Valid Window

```
left = 0
state = 0
best = float('inf')
for right in range(len(nums)):
    state += nums[right]
    while valid(state):
        best = min(best, right - left + 1)
        state -= nums[left]
        left += 1
```

At Most K

```
left = 0
count = {}
best = 0
for right in range(len(nums)):
    count[nums[right]] = count.get(nums[right], 0) + 1
    while len(count) > k:
        count[nums[left]] -= 1
        if count[nums[left]] == 0:
            del count[nums[left]]
        left += 1
    best = max(best, right - left + 1)
```

Exactly K

```
def exactly(nums, k):
    return at_most(nums, k) - at_most(nums, k - 1)
```

Counting Subarrays

```
left = 0
count = {}
total = 0
for right in range(len(nums)):
    count[nums[right]] = count.get(nums[right], 0) + 1
    while invalid(count):
        count[nums[left]] -= 1
        if count[nums[left]] == 0:
            del count[nums[left]]
        left += 1
    total += right - left + 1
```

Minimum Window (frequency matching)

```
need = Counter(t)
required = len(need)
window = {}
have = 0
left = 0
best_len, best_left = float('inf'), 0

for right in range(len(s)):
    c = s[right]
    window[c] = window.get(c, 0) + 1
    if c in need and window[c] == need[c]:
        have += 1
    while have == required:
        if right - left + 1 < best_len:
            best_len, best_left = right - left + 1, left
        window[s[left]] -= 1
        if s[left] in need and window[s[left]] < need[s[left]]:
            have -= 1
        left += 1
```